RK2206实训箱操作手册—VEML7700开发

VEML7700光照传感器

1. 目录

- 实验内容
- VEML7700 API说明
  - 函数说明
    - VEML7700初始化
    - VEML7700配置ALS参数
    - VEML7700设置省电模式
    - VEML7700读取环境光数据
  - 常量和枚举类型说明
    - I2C 端口
    - I2C 地址
    - ALS增益设置 (als_gain_t)
    - ALS积分时间设置 (als_it_t)
    - ALS持续保护数设置 (als_pers_t)
    - ALS中断使能设置 (als_int_en_t)
    - ALS关机设置 (als_sd_t)
    - 省电模式设置 (psm_mode_t)
    - 省电使能设置 (psm_en_t)
  - 寄存器地址定义
- 代码编译
  - 修改文件
  - 添加文件
  - 修改 BUILD.gn 文件
  - 编译结果
  - 烧录固件
  - 运行结果
- 注意事项
- 技术规格

2. 实验内容

在RK2206实训箱上使用I2C接口，实现VEML7700光照传感器的编程开发。

3. VEML7700 API说明

I2C相关API详见PPT。

3.1 函数说明

3.1.1 (一)、VEML7700初始化

定义:	veml7700_init(void)
功能:	初始化VEML7700环境光传感器模块
参数:	void
返回值:	void
依赖:	\code_base\b9_i2c_veml7700\include\veml7700.h
Tips:	该接口只需调用一次，不支持重复调用。内部会初始化I2C0_M2端口，频率为100KHz

Table 1

3.1.2 (二)、VEML7700配置ALS参数

定义:	veml7700_set_als_conf_0(als_gain_t gain, als_it_t it, als_pers_t pers, als_int_en_t int_en, als_sd_t sd)
功能:	配置VEML7700的ALS（环境光传感器）参数
参数:	`gain`:增益设置(ALS_GAIN_1/2/3/4); `it`:积分时间(25ms-800ms); `pers`:持续保护数(1/2/4/8); `int_en`:中断使能; `sd`:关机设置
返回值:	void
依赖:	\code_base\b9_i2c_veml7700\include\veml7700.h
Tips:	参数可查看头文件中的枚举定义，用于配置传感器的增益、积分时间、中断等参数

Table 2

3.1.3 (三)、VEML7700设置省电模式

定义:	veml7700_set_psm(psm_mode_t psm_mode, psm_en_t psm_en)
功能:	设置VEML7700的省电模式
参数:	`psm_mode`:省电模式(PSM_MODE_1/2/3/4); `psm_en`:省电使能(PSM_DISABLE/PSM_ENABLE)
返回值:	void
依赖:	\code_base\b9_i2c_veml7700\include\veml7700.h
Tips:	省电模式可降低功耗，但会影响刷新时间，具体时间可参考数据手册

Table 3

3.1.4 (四)、VEML7700读取环境光数据

定义:	**veml7700_read_data(double *dat)**
功能:	**读取VEML7700的环境光强度数据**
参数:	`dat`**:指向存储光强度数据的double型指针**
返回值:	**void**
依赖:	**\code_base\b9_i2c_veml7700\include\veml7700.h**
Tips:	**读取的数据已转换为实际光强度值(单位:lux)，转换系数为2.1504**

Table 4

3.2 常量和枚举类型说明

3.2.1 I2C 端口

- `EI2C0_M2`

3.2.2 I2C 地址

- `0x10`

3.2.3 ALS增益设置 (als_gain_t)

- `ALS_GAIN_1`: 增益 x 1
- `ALS_GAIN_2`: 增益 x 2
- `ALS_GAIN_3`: 增益 x (1/8)
- `ALS_GAIN_4`: 增益 x (1/4)

3.2.4 ALS积分时间设置 (als_it_t)

- `ALS_IT_25MS`: 25ms
- `ALS_IT_50MS`: 50ms
- `ALS_IT_100MS`: 100ms
- `ALS_IT_200MS`: 200ms
- `ALS_IT_400MS`: 400ms
- `ALS_IT_800MS`: 800ms

3.2.5 ALS持续保护数设置 (als_pers_t)

- `ALS_PERS_1`: 1次
- `ALS_PERS_2`: 2次
- `ALS_PERS_4`: 4次
- `ALS_PERS_8`: 8次

3.2.6 ALS中断使能设置 (als_int_en_t)

- `ALS_INT_DISABLE`: 中断禁用
- `ALS_INT_ENABLE`: 中断使能

3.2.7 ALS关机设置 (als_sd_t)

- `ALS_SD_POWERON`: 开机
- `ALS_SD_SHUTDOWN`: 关机

3.2.8 省电模式设置 (psm_mode_t)

- `PSM_MODE_1`: 模式1
- `PSM_MODE_2`: 模式2
- `PSM_MODE_3`: 模式3
- `PSM_MODE_4`: 模式4

3.2.9 省电使能设置 (psm_en_t)

- `PSM_DISABLE`: 省电禁用
- `PSM_ENABLE`: 省电使能

3.3 寄存器地址定义

寄存器名称	地址	功能描述
`VEML7700_ALS_CONF_0`	0x00	ALS增益、积分时间、中断和关机设置
`VEML7700_ALS_WH`	0x01	ALS高阈值窗口设置
`VEML7700_ALS_WL`	0x02	ALS低阈值窗口设置
`VEML7700_POWER_SAVING`	0x03	省电模式设置
`VEML7700_ALS`	0x04	ALS数据(16位)
`VEML7700_WHITE`	0x05	WHITE数据(16位)
`VEML7700_ALS_INT`	0x06	ALS中断触发事件

Table 5

4. 代码编译

4.1 修改文件

因I2C API中`IoTI2cWriteRead`函数并无实例，需修改`b9_i2c_veml7700/src/veml7700.c`中的`veml7700_read_data`函数

更改为如下:

```
void veml7700_read_data(double *dat)
{
```

```
    // uint8_t send_data[]  = {VEML7700_ALS};
    uint8_t read_data[2] = {0};
    // IoTI2cWriteRead(VEML7700_I2C_PORT, VEML7700_I2C_ADDRESS, send_data,
sizeof(send_data), read_data, sizeof(read_data));
    uint8_t reg_addr = VEML7700_ALS;
    uint32_t ret;
    ret = LzI2cReadReg(VEML7700_I2C_PORT, VEML7700_I2C_ADDRESS, &reg_addr, 1,
read_data, 2);
    if (ret != LZ_HARDWARE_SUCCESS) {
        printf("i2c read reg fail!\r\n");
        return;
    }
    uint16_t u16_data = (read_data[1] << 8) | read_data[0];
    *dat              = (double)u16_data * 2.1504;
}"
```

Fence 1

```
void veml7700_read_data(double *dat)
{
    // uint8_t send_data[]  = {VEML7700_ALS};
    uint8_t read_data[2] = {0};
    // IoTI2cWriteRead(VEML7700_I2C_PORT, VEML7700_I2C_ADDRESS, send_data, sizeof(send_data), read_data, sizeof(read_data));
    uint8_t reg_addr = VEML7700_ALS;
    uint32_t ret;
    ret = LzI2cReadReg(id: VEML7700_I2C_PORT, slaveAddr: VEML7700_I2C_ADDRESS, regAddr: &reg_addr, regLen: 1, data: read_data, len: 2);
    if (ret != LZ_HARDWARE_SUCCESS) {
        printf("i2c read reg fail!\r\n");
        return;
    }
    uint16_t u16_data = (read_data[1] << 8) | read_data[0];
    *dat              = (double)u16_data * 2.1504;
}
```

不可用

Figure 1

4.2 添加文件

将`~vendor/company_demo/b9_i2c_veml7700`这整个文件夹拖入`~/application/samples`路径下

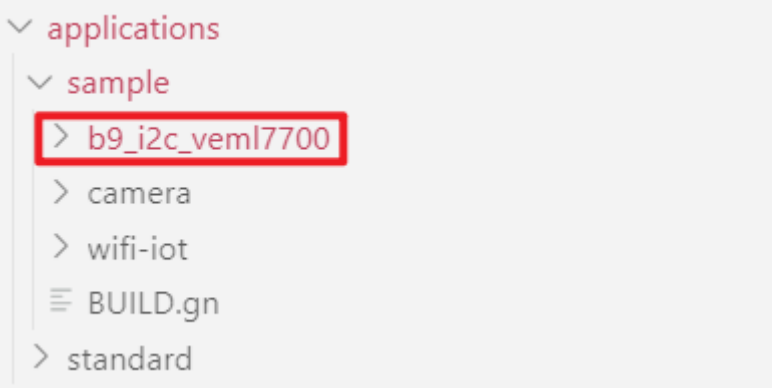


Figure 2

4.3 修改 BUILD.gn 文件

修改 `~/application/samples/` 路径下 BUILD.gn 文件，将如下代码添加到对应位置，指定 `veml7700_example` 参与编译。

```
    "./b5_i2c_sht40:veml7700_example",
```

Fence 2

```
import("//build/lite/config/component/lite_component.gni")

lite_component("app") {
    features = [
        "./b9_i2c_veml7700:veml7700_example",
    ]

    deps = [
    ]
}
```

Figure 3

修改 `device/lockzhiner/rk2206/sdk_liteos` 路径下 Makefile 文件，添加 `-lveml7700_example` 参与编译。

```
hardware_LIBS = -lhal_iothardware -lhardware -lshellcmd -lveml7700_example
```

Fence 3

```
#######################################
# LDFLAGS
#######################################
LDSCRIPT = board.ld

boot_LIBS = -lbootstrap -lbroadcast
hardware_LIBS = -lhal_iothardware -lhardware -lshellcmd -lveml7700_example
hdf_LIBS = -lhdf_config -lhdf_core -lhdf_osal_lite -lhdf_platform_lite \
#-lsample_driver -lgpio_driver -lpwm_driver -li2c_driver -ladc_driver -lspi_driver
```

Figure 4

4.4 编译结果

```
root@05fdda9d5ead: /home/
[OHOS INFO] [829/851] AR libs/libfatfs.a
[OHOS INFO] [830/851] STAMP obj/base/security/deviceauth/frameworks/deviceauth_lite/hichainsdk.stamp
[OHOS INFO] [831/851] STAMP obj/kernel/liteos_m/components/fs/fs.stamp
[OHOS INFO] [832/851] AR libs/libshell.a
[OHOS INFO] [833/851] gcc cross compiler obj/third_party/paho_mqtt/MQTTClient-C/src/libpahomqtt_static.MQTTClient.o
[OHOS INFO] [834/851] AR libs/libpahomqtt_static.a
[OHOS INFO] [835/851] AR libs/liblwip.a
[OHOS INFO] [836/851] STAMP obj/kernel/liteos_m/kernel.stamp
[OHOS INFO] [837/851] ACTION //test/xts/acts/build_lite:acts_xdevice(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [838/851] STAMP obj/test/xts/acts/build_lite/acts_xdevice.stamp
[OHOS INFO] [839/851] ACTION //test/xts/acts/build_lite:acts_generate_module_data(//build/lite/toolchain:arm-none-eabi-g
cc)
[OHOS INFO] [840/851] STAMP obj/test/xts/acts/build_lite/acts_generate_module_data.stamp
[OHOS INFO] [841/851] ACTION //vendor/lockzhiner/rk2206/hdf_config:hc_gen(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [842/851] STAMP obj/vendor/lockzhiner/rk2206/hdf_config/hc_gen.stamp
[OHOS INFO] [843/851] gcc cross compiler obj/out/rk2206/lockzhiner-rk2206/gen/vendor/lockzhiner/rk2206/hdf_config/libhdf
_config.hdf_hex.o
[OHOS INFO] [844/851] AR libs/libhdf_config.a
[OHOS INFO] [845/851] STAMP obj/device/rockchip/rk2206/sdk_liteos/sdk.stamp
[OHOS INFO] [846/851] STAMP obj/device/rockchip/rk2206/sdk_liteos/wifiiot_sdk.stamp
[OHOS INFO] [847/851] ACTION //test/xts/acts/build_lite:acts(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [848/851] STAMP obj/test/xts/acts/build_lite/acts.stamp
[OHOS INFO] [849/851] STAMP obj/build/lite/ohos.stamp
[OHOS INFO] [850/851] ACTION //device/rockchip/rk2206/sdk_liteos:liteos(//build/lite/toolchain:arm-none-eabi-gcc)
[OHOS INFO] [851/851] STAMP obj/device/rockchip/rk2206/sdk_liteos/liteos.stamp
[OHOS INFO] /home/openharmony/vendor/lockzhiner/rk2206/fs.yml not found, stop packing fs. If the product does not need t
o be packaged, ignore it.
[OHOS INFO] lockzhiner-rk2206 build success
[OHOS INFO] cost time: 0:01:39
root@05fdda9d5ead:/home/openharmony#
```

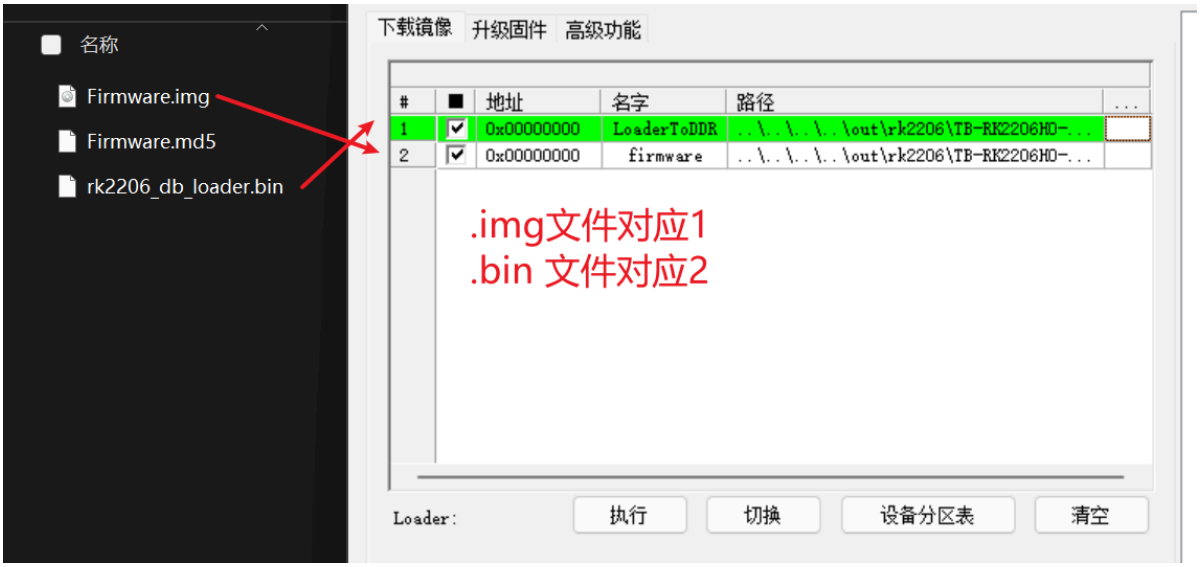
Figure 5

4.5 烧录固件

打开烧录工具RKDevTool

Figure 6

指定两个烧录文件, 编译出的目标文件在 `out/rk2206/lockzhiner-rk2206/images` 目录下。



在RK2206实训箱上上，长按左侧模块控制终端的MaskRom按键（MaskRom按键不要松开），点击ReSet按键（短按，按下后松开，但MaskRom按键不要松开，大约5秒后松开)，烧写工具出现：“发现一个MASKROM设备”。如下图所示：

Figure 7

此时点击“执行”，下载程序，下载完成如下：

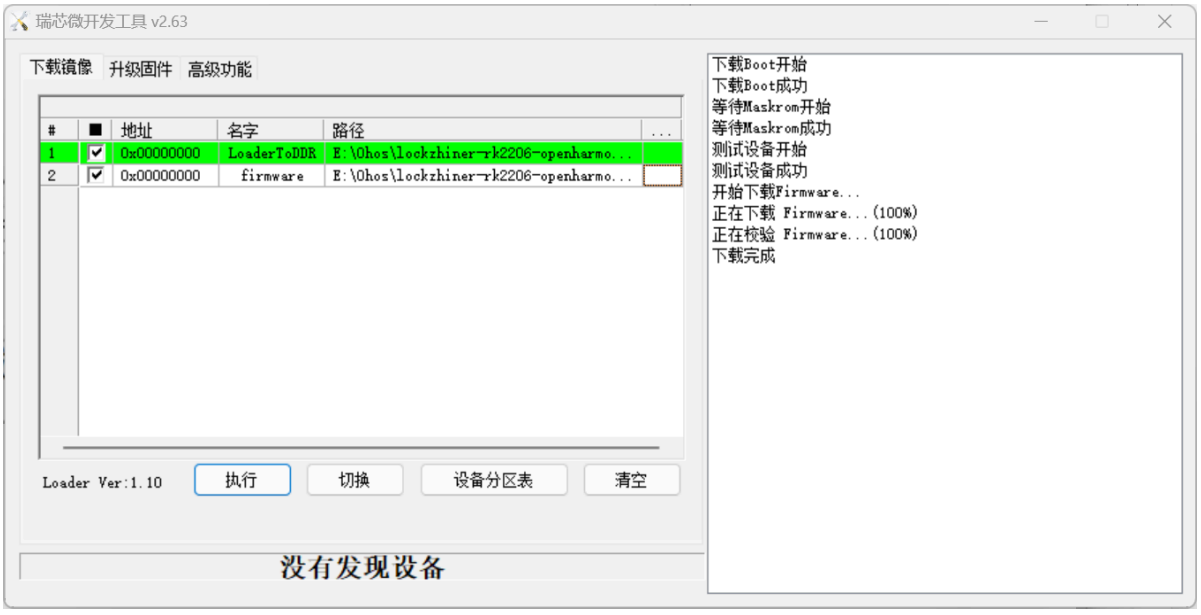


Figure 8

4.6 运行结果

示例代码编译烧录代码后，按下开发板的RESET按键，连接串口，可以看到光照强度信息。

```
entering kernel init...
hilog will init.
```

```
[MAIN:D]Main: LOS_Start ...
Entering scheduler
OHOS # hiview init success.
[GPIO:D]LzGpioInit: id 1 is initialized successfully
[GPIO:D]LzGpioInit: id 0 is initialized successfully
light_intensity: 0.00
light_intensity: 365.57
light_intensity: 365.57
light_intensity: 365.57
light_intensity: 365.57
light_intensity: 365.57
light_intensity: 365.57
light_intensity: 365.57
light_intensity: 354.82
light_intensity: 359.12
light_intensity: 359.12
light_intensity: 356.97
light_intensity: 361.27
light_intensity: 116.12
```

Fence 4

```
entering kernel init...
hilog will init.
[MAIN:D]Main: LOS_Start ...
Entering scheduler
OHOS # hiview init success.[GPIO:D]LzGpioInit: id 1 is initialized successfully
[GPIO:D]LzGpioInit: id 0 is initialized successfully
veml7700 init succeed!
light_intensity: 286.00
light_intensity: 288.15
light_intensity: 283.85
light_intensity: 286.00
light_intensity: 288.15
light_intensity: 283.85
light_intensity: 286.00
light_intensity: 262.35
light_intensity: 245.15
light_intensity: 234.39
light_intensity: 62.36
light_intensity: 6.45
light_intensity: 6.45
light_intensity: 283.85
light_intensity: 286.00
light_intensity: 283.85
```

存在遮挡时

Figure 9

5. 注意事项

1. **初始化顺序**: 必须先调用`veml7700_init()`进行初始化，再进行其他操作
2. **I2C通信**: 传感器使用I2C0_M2端口，地址为0x10，通信频率100KHz
3. **数据转换**: 读取的原始数据会自动转换为lux单位，转换系数为2.1504
4. **参数配置**: 建议根据实际应用场景选择合适的增益和积分时间参数

5. **省电模式**: 启用省电模式会降低功耗，但会增加数据刷新时间
6. **编译问题**: 核查有无进行3.1步骤

6. 技术规格

- **通信接口**: I2C
- **I2C地址**: 0x10
- **测量范围**: 0-120k lux
- **分辨率**: 16位
- **工作电压**: 2.5V-3.6V
- **工作温度**: -25°C ~ +85°C